GESTIÓN DE CONFIGURACIÓN Y MANTENIMIENTO DE SOFTWARE · UNIDAD 1

# Instrumentos F-01 a F-07

Evolución del software y mantenimiento · Maestría en Ingeniería de Software, Universidad de La Sabana

Estos siete formatos preparan las secciones del informe de la Actividad 1.2. Se llenan en grupo, en el orden en que aparecen, sobre el proyecto de código abierto que el grupo eligió. **No se suben a la plataforma**: son el borrador de las secciones del informe y la evidencia que el grupo debe poder mostrar si se le pregunta de dónde sale una afirmación.

| Formato | Qué prepara | Sección del informe | Evidencia que usa |
|---|---|---|---|
| **F-01** Ficha del sistema | Qué sistema se estudia, por qué cumple los criterios y qué parte se mide | 1. Caracterización | Repositorio, sitio del proyecto, registro de cambios |
| **F-02** Evolución por versión | Tamaño, crecimiento y actividad de cada versión | 2. Métricas de evolución | `evolucion_por_version.csv` |
| **F-03** Clasificación de cambios | Qué tipo de mantenimiento consume el esfuerzo | 3. Perfil de mantenimiento | `muestra_cambios.csv` y el diff de cada commit |
| **F-04** Contraste con las leyes de Lehman | Qué leyes se observan, con evidencia y contraevidencia | 4. Leyes de la evolución | F-02, F-03 y registro de cambios |
| **F-05** Etapa del ciclo de vida | En qué etapa del modelo por etapas está el sistema | 5. Etapa y pronóstico | F-02 a F-04 |
| **F-06** Registro de deuda técnica | Cinco deudas con su origen, costo y prioridad | 6. Deuda técnica | Código, issues, registro de cambios |
| **F-07** Verificación previa | Que cada afirmación del informe tenga respaldo | Antes de entregar | El informe completo |

**EL ORDEN IMPORTA**

Primero se mide (F-02) y se clasifica (F-03); después se contrasta con las leyes (F-04). Un grupo que decide antes qué leyes se cumplen y luego busca los datos que lo confirman suele encontrarlos, y el informe pierde valor.

## Ficha del sistema F-01

Identifica el sistema y deja por escrito por qué sirve para estudiar su evolución. Un proyecto que no cumple los criterios produce un informe sin evidencia suficiente, por bueno que sea el análisis.

| | |
|---|---|
| **Nombre del proyecto y enlace al repositorio** | |
| **Qué hace el sistema, en dos líneas, y quién lo usa** | |
| **Lenguaje principal y licencia** | |
| **Primera versión publicada (fecha) y última versión (fecha)** | |
| **Número de versiones con funcionalidad nueva (patrón de etiquetas usado)** | |
| **Qué se mide como código fuente (rutas incluidas) y qué se deja fuera (pruebas, documentación, ejemplos)** | |
| **Dónde está el registro de cambios (CHANGES, CHANGELOG, notas de versión)** | |
| **Cambios de estructura conocidos (renombres de carpetas, divisiones, migraciones de lenguaje)** | |
| **Tipo de programa según Lehman (S, P o E) y por qué** | |

### Criterios que debe cumplir el proyecto

| Criterio | Cumple | Evidencia |
|---|---|---|
| Historia pública de al menos cinco años en Git | | |
| Al menos ocho versiones con funcionalidad nueva marcadas con etiquetas | | |
| Registro de cambios mantenido por el proyecto | | |
| Gestor de issues público | | |

| Criterio | Cumple | Evidencia |
| --- | --- | --- |
| Tamaño manejable: la herramienta de métricas termina en menos de diez minutos | | |
| Ningún otro grupo del curso eligió el mismo proyecto | | |

## Evolución por versión F-02

Se llena con `evolucion_por_version.csv`, que produce la herramienta `herramientas/metricas_evolucion.py`. La tabla del informe puede resumir las versiones; los gráficos deben usar todas.

| Versión | Fecha | Archivos | Líneas | Commits desde la anterior | Autores | Archivos modificados | Líneas agregadas / eliminadas |
|---|---|---|---|---|---|---|---|
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |

### Gráficos mínimos

1. **Tamaño en el tiempo:** líneas de código fuente por versión, con la fecha en el eje horizontal. Sirve para la ley VI.
2. **Crecimiento por versión:** líneas agregadas y eliminadas entre versiones consecutivas. Sirve para la ley V.
3. **Actividad:** commits y autores entre versiones. Sirve para las leyes III y IV.

| | |
|---|---|
| **Tres observaciones sobre los datos, sin interpretarlos todavía** | |
| **Versiones atípicas (saltos, caídas) y qué dice el registro de cambios sobre ellas** | |
| **Limitaciones de la medición (qué cuentan y qué no cuentan las líneas)** | |

## Clasificación de cambios F-03

Se llena con `muestra_cambios.csv`. Cada commit se clasifica **leyendo el diff** (`git show <commit>`), no el mensaje. La columna `sugerencia_heuristica` se usa como punto de partida y se anota cuándo acierta y cuándo no.

| Tipo | Pregunta que lo decide |
|---|---|
| **Correctivo** | ¿Corrige un comportamiento que ya debía funcionar y no funcionaba? |
| **Adaptativo** | ¿Responde a un cambio del entorno: lenguaje, dependencias, plataforma, normas, navegadores? |
| **Perfectivo** | ¿Agrega o mejora funcionalidad, rendimiento o usabilidad que nadie reportó como falla? |
| **Preventivo** | ¿Reduce la dificultad de cambios futuros sin cambiar el comportamiento: refactorización, pruebas, retiro de código obsoleto? |
| **Fuera del producto** | ¿Toca solo documentación, publicación o infraestructura del proyecto? |

| Commit | Asunto | Heurística | Clasificación | Evidencia en el diff |
|---|---|---|---|---|
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |
| | | | | |

La muestra completa (40 commits como mínimo) va en un anexo del informe o en una hoja de cálculo enlazada. En el cuerpo del informe va la distribución por tipo y los casos difíciles.

| | |
|---|---|
| **Distribución por tipo (número y porcentaje)** | |
| **Porcentaje de acierto de la heurística y en qué tipos falla más** | |
| **Tres casos difíciles: por qué admitían dos tipos y cuál eligió el grupo** | |

## Contraste con las leyes de Lehman F-04

Para cada ley se escribe lo que se esperaría ver en los datos si la ley se cumple, lo que efectivamente se ve y la contraevidencia. El informe debe contrastar al menos **cuatro leyes**, entre ellas la II y la VI.

| Ley | Qué se esperaría ver | Evidencia (F-02, F-03, registro de cambios) | Contraevidencia | Veredicto |
|---|---|---|---|---|
| **I** Cambio continuo | | | | |
| **II** Complejidad creciente | | | | |
| **III** Autorregulación | | | | |
| **IV** Estabilidad organizacional | | | | |
| **V** Conservación de la familiaridad | | | | |
| **VI** Crecimiento continuo | | | | |
| **VII** Calidad decreciente | | | | |
| **VIII** Sistema de retroalimentación | | | | |

Veredicto: **se observa**, **se observa con matices**, **no se observa** o **los datos no alcanzan**. «Los datos no alcanzan» es una respuesta válida si se explica qué dato haría falta.

## Etapa del ciclo de vida F-05

Ubica el sistema en el modelo por etapas de Rajlich y Bennett (2000): desarrollo inicial, evolución, servicio, retiro gradual y cierre. La etapa se decide con evidencia; la sensación de que un proyecto «está maduro» no basta.

| Etapa | Señales que la indican | Señales presentes en el proyecto |
|---|---|---|
| **Desarrollo inicial** | Primera versión funcional; arquitectura y conocimiento del equipo se están formando | |
| **Evolución** | Funcionalidad nueva frecuente; cambios de estructura posibles; el equipo conoce la arquitectura | |
| **Servicio** | Predominan correcciones y adaptaciones pequeñas; cambios de estructura evitados o costosos | |
| **Retiro gradual** | Solo correcciones críticas; se anuncia reemplazo o fin de soporte | |
| **Cierre** | Repositorio archivado; usuarios migrados | |

| | |
|---|---|
| **Etapa en la que está el sistema hoy** | |
| **Momento (versión) en que cambió de etapa, si lo hubo, y la evidencia** | |
| **Qué tendría que pasar para que pasara a la etapa siguiente** | |

## Registro de deuda técnica F-06

Cinco deudas concretas, cada una con un lugar en el código o en el proyecto. Para el origen se usa el cuadrante de Fowler (2009): **deliberada o inadvertida**, **prudente o imprudente**.

| # | Deuda y dónde está | Origen | Interés: qué cuesta mantenerla | Pago: qué cuesta quitarla | Evidencia | Prioridad |
|---|---|---|---|---|---|---|
| 1 | | | | | | |
| 2 | | | | | | |
| 3 | | | | | | |
| 4 | | | | | | |
| 5 | | | | | | |

DÓNDE BUSCAR

- Código marcado como obsoleto: `git grep -n -i deprecat <versión> -- <ruta>`
- Supresiones de advertencias: `type: ignore`, `noqa`, `@SuppressWarnings`, `eslint-disable`
- Comentarios `TODO`, `FIXME`, `HACK`: `git grep -n -E "TODO|FIXME|HACK"`
- Issues abiertos hace más de dos años con etiquetas como *refactor*, *cleanup* o *tech debt*
- Entradas del registro de cambios que retiran código, capas de compatibilidad o dependencias

## Verificación previa a la entrega F-07

El grupo la recorre completa antes de subir el informe. Cada «no» se corrige o se explica en el informe.

| Verificación | Sí / No |
|---|---|
| El proyecto cumple los seis criterios del F-01 y el informe dice cómo se verificaron | |
| El informe dice qué rutas se midieron como código fuente y por qué | |
| Los tres gráficos del F-02 están en el informe, con título, ejes y fuente | |
| Las versiones atípicas se explican con el registro de cambios | |
| La muestra tiene al menos 40 commits, la semilla está declarada y cualquiera puede repetirla | |
| Cada commit de la muestra se clasificó leyendo el diff | |
| Se contrastan al menos cuatro leyes, entre ellas la II y la VI, con evidencia y contraevidencia | |
| La etapa del F-05 se sostiene con señales concretas | |
| Las cinco deudas tienen un lugar en el código o en el proyecto, no son genéricas | |
| Las tres recomendaciones se derivan de las deudas y de la etapa, y están priorizadas | |
| El informe tiene máximo 15 páginas y sigue la norma APA 7 | |
| Los CSV y el comando exacto con que se generaron van como anexo o enlace | |